

CASE STUDY · FULL-STACK DEVELOPMENT

ShopEase

A Production-Style Multi-Vendor E-Commerce Platform
Built on the MERN Stack

Muhammad Haroon Khan

BSCS (7th Semester), University of Mianwali · Dev Weekends '26 Fellowship
September 2026

1. Overview

ShopEase is a full-stack, multi-vendor e-commerce marketplace where independent sellers can register, get approved, and manage their own storefronts, while buyers browse products from many sellers in one unified shopping experience. The platform was built end-to-end — authentication, product management, a real Stripe-powered checkout, a multi-vendor order system, seller and admin dashboards, and a review system — following a structured, phase-by-phase build process rather than a single monolithic build.

This project was completed as the flagship assignment of the Dev Weekends '26 fellowship, chosen for its scope: it exercises nearly every layer of a real-world web application, from database schema design to third-party payment integration to production deployment.

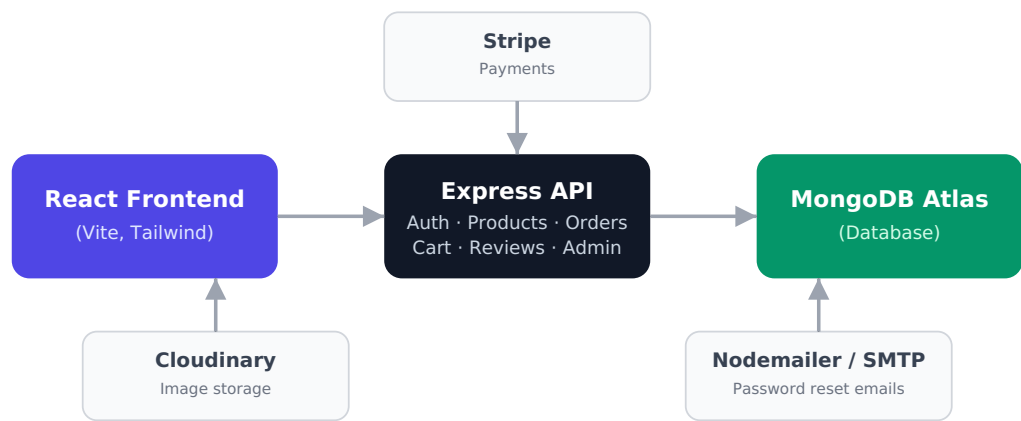
2. Goals of the Project

- Build a genuinely functional multi-role authentication system (buyer, seller, admin) — not just a UI mockup
- Implement a real, working payment flow using Stripe's test-mode API, not a simulated "fake checkout"
- Correctly handle the hardest part of multi-vendor commerce: splitting a single cart into per-seller orders when a buyer purchases from multiple sellers at once
- Give sellers and admins dashboards with real operational value (approvals, order status updates, platform-wide stats)
- Produce a portfolio-quality deliverable: a live deployed URL, a clean GitHub repository, and this case study

3. System Architecture Overview

The application follows a standard decoupled MERN architecture: a React (Vite) single-page frontend communicates with an Express/Node.js REST API, backed by MongoDB (Atlas) for persistence. Three external services are integrated directly into the backend: **Stripe** for payment processing, **Cloudinary** for product image storage, and **Nodemailer** (via SMTP) for transactional password-reset emails.

4. System Architecture Diagram



5. Key Features

5.1 Multi-Role User System

Three distinct roles, each with its own permissions, enforced by JWT-based auth middleware:

Role	Capabilities
Buyer	Browse/search products, manage cart, checkout via Stripe, track orders, leave verified-purchase reviews
Seller	Register with a shop profile (pending admin approval), list/edit/delete products with image upload, manage incoming orders
Admin	Approve/revoke sellers, view all products and orders platform-wide, view aggregate revenue and growth stats

5.2 Product Management & Shopping

- Product CRUD with multi-image upload (Cloudinary), stock tracking, and optional discount pricing
- Keyword search and category filtering, with server-side pagination
- Persistent server-side cart tied to the buyer's account

5.3 Payments & Order Lifecycle

- Real Stripe PaymentIntent creation and client-side card confirmation (test mode)
- Server verifies payment success directly with Stripe before creating any order — never trusts the client alone
- **Multi-vendor order splitting:** a single checkout is automatically divided into one order per seller, linked by a shared order-group ID, so each seller only ever sees their own slice while the buyer sees one coherent purchase
- Order status lifecycle: Processing → Shipped → Delivered (or Cancelled), updatable by the owning seller or admin

5.4 Reviews & Trust

- Verified-purchase-only reviews — a buyer must have a paid order containing the product
- One review per buyer per product; product rating auto-recalculates on every review add/remove

5.5 Authentication Hardening

- Forgot/reset password flow with time-limited, hashed reset tokens emailed via Nodemailer
- Change-password flow for logged-in users
- Password visibility toggle across every password field in the UI

JWT Auth

Role-Based Access Control

Stripe Payments

Cloudinary Uploads

MongoDB Aggregation

Responsive UI

6. Tech Stack

Frontend

- React 18 (Vite)
- React Router
- Tailwind CSS
- Axios
- Stripe.js / React Stripe.js
- lucide-react (icons)

Backend

- Node.js / Express.js
- MongoDB Atlas + Mongoose
- JSON Web Tokens (JWT) + bcrypt
- Multer + Cloudinary (image uploads)
- Stripe API (payments)
- Nodemailer (transactional email)

Deployment

- Frontend & backend both deployed on Vercel
- Database hosted on MongoDB Atlas (cloud)

7. Challenges & Solutions

CHALLENGE — SPLITTING ORDERS ACROSS SELLERS

A single cart can contain products from different sellers, but each seller needs to manage only their own orders. **Solution:** on checkout, cart items are grouped by seller in-memory, and one Order document is created per seller, all sharing a common orderGroupId — giving buyers a unified view while keeping seller data properly isolated.

CHALLENGE — TRUSTING CLIENT-REPORTED PAYMENT SUCCESS

A malicious client could claim a payment succeeded without actually paying. **Solution:** the backend never trusts the client's word — it re-verifies the PaymentIntent's status directly with Stripe's API before creating any order or decrementing stock.

CHALLENGE — SOURCING ACCURATE DEMO PRODUCT IMAGES

Early seed data used random stock-photo pools, which produced mismatched images (e.g. a houseplant photo on a "Wall Art" listing). **Solution:** the seed script was rebuilt to query the Pexels image-search API live, using a clean search term per product name, so each demo product gets a genuinely matching photo rather than a manually-guessed one.

CHALLENGE — KEEPING A GROWING UI PROFESSIONAL UNDER DEADLINE PRESSURE

As scope grew (auth, cart, dashboards, admin panel), it was easy for the UI to default to plain, unstyled markup. **Solution:** the interface was iterated deliberately — a distinct dark navbar, consistent brand color, hover/transition polish on product cards, and a proper footer — rather than shipping default browser styling.

8. Results

- A fully functional multi-vendor marketplace with real payment processing, not a mock checkout
- Three working role-based dashboards (buyer shopping flow, seller product/order management, admin oversight)
- A seeded catalog of 52 realistic products across three categories, used to validate search, filtering, and pagination under realistic data volume
- A codebase organized into clear, independently testable phases (auth → products → cart/payments → admin/reviews → frontend → polish → deployment)

9. Lessons Learned

- Payment flows need server-side verification at every step — client-reported state is never sufficient for anything involving money
- Multi-vendor data models are meaningfully more complex than single-vendor ones; designing the Order schema around "one document per seller" early avoided a much messier retrofit later
- Placeholder/demo data deserves the same care as production data — mismatched or broken demo content undermines trust in an otherwise solid build
- Incremental, phase-based delivery (backend phase by phase, then frontend phase by phase) made a large scope manageable and easy to test along the way

10. Future Improvements

- Seller payout automation (currently, payments settle to a single platform Stripe account)
- Real-time order notifications (WebSockets) for sellers
- Wishlist and product recommendation features
- Automated test coverage (unit + integration tests) for the API layer

11. Links

LIVE APPLICATION

[Add your deployed Vercel URL here]

GITHUB REPOSITORY

[Add your GitHub repo URL here]